MLFF campaign storage and I/O management specification

Status: Accepted current normative contract for the owner-driven storage subsystem. Historical STOR1-STOR5 design notes are archived in `docs/history/mlff/STOR1_STOR5_HISTORICAL_DESIGN.md`, and the retired DATA9B4 storage/restart contract is archived under `docs/history/mlff/retired_specs/`. Neither is current authority.

Purpose

This specification defines how the MLFF campaign subsystem manages persistent storage and I/O. Storage is strictly subordinate to the accepted P1-P7 scientific and currentness owners: it may change representation, retention, caching, deduplication, archival, recovery, admission, and I/O execution mechanics, and it may never change what the campaign means.

1. Non-negotiable protections

1. **Storage is never a second scientific authority.** It does not decide scientific identity, target membership, selected size, cross-validation acceptance, representative checkpoint choice, final publication membership, qualification outcome, locked activation, or a release verdict. It reads what the owners say.
2. **External inputs are indestructible.** User-supplied source datasets, training sets, replay trajectories, foundation models, and true-label reference material outside the campaign workspace are read-only regardless of path, symlink, configuration, or record reference.
3. **A reference is not an authority.** A path appearing in TOML, SQLite, JSON, a manifest, or an archive manifest confers no deletion authority. Every mutation additionally passes physical containment and ownership validation.
4. **Symlink targets are never traversed.** A campaign-owned symlink object may be unlinked; its target is never followed for traversal or deletion.
5. **Ambiguity retains.** Corrupt, unreadable, or unclassifiable owner state retains the affected artifacts until ownership is repaired or they are independently proven disposable.
6. **A report never authorizes a mutation.** Both report modes are read-only and advisory.
7. **Retired authority stays retired.** STOR1-STOR5 tier policy, `workspace/runs` conventions, `active_process.json`, PID/age/pathname rules, the retired `evaluate`/`verify` stages, DATA7/DATA8 stage folklore, and SELECT2 never regain destructive authority.
8. **Only the current invocation authorizes.** Authority to mutate comes from `--apply` on the invocation being run, and the action comes from the subcommand being run. Persisted configuration cannot carry either: an `apply` or `action` key under `[storage]` is rejected outright rather than honored or silently ignored, and no environment variable is read.
9. **Containment is not ownership.** Being beneath an owner's directory does not make a file that owner's. A directory artifact is destructively recursed into only under an explicit closed-subtree contract from the real owner; otherwise only individually certified children participate and every unexpected descendant is retained.
10. **A mount boundary is an ownership boundary.** A filesystem mounted below an authorized root exposes externally owned bytes through a campaign-looking path. Traversal stops there, and if the platform cannot answer the question the artifact is retained.

2. The one consequential flow

Every consequential storage mutation converges on a single path:

```
real P1-P7 owners
 -> owner views
 -> cross-owner inventory snapshot (transitive protection closure)
 -> one canonical resolved storage policy
 -> immutable owner-bound plan
```

```
-> explicit operator authorization
-> owner-local publication barrier + owner/currentness/filesystem revalidation
-> executor
-> durable audit
-> restart-equivalent product
```

There is exactly one destructive implementation. Reporting, planning, and execution are separate, and only execution mutates.

3. Retention is a cross-owner closure

Eligibility is computed over the **transitive dependency closure of every current or restartable owner**, not by asking each artifact's nominal owner in isolation.

Normative consequences:

- the current P7 publication pins the exact P5 published member checkpoint bytes for as long as that publication resolves and authenticates as current, **including after the P7 attempt retention reference has been released**;
- the current P4 terminal/selected authority pins the P3 immutable evidence its canonical loader and reconciliation chain require;
- a truthful `waiting_for_reference` pins every predecessor artifact needed to resume the exact frozen publication and qualification lineage, including the frozen external-reference request;
- protection is monotone: no owner's cache or history classification can override another current owner's requirement;
- historical evidence becomes archive- or reclamation-eligible only after no current or restartable descendant needs its hot representation;
- the closure is derived from current owner records on every invocation. There is no second persistent dependency database;
- the owner graph is checked for integrity before any consequential planning: owner identities must be unique and every declared dependency edge must resolve to a real owner view. An incomplete or contradictory graph refuses consequential planning instead of planning against a closure it cannot compute.

4. Race-safe mutation

A recent snapshot is not an authorization. P5 and P7 both publish an immutable object and then publish the current pointer that makes it current, so there is a legitimate window in which the object exists and nothing references it.

- Each owner exposes a **publication barrier** for one generation. The publisher holds it across object publication and pointer commit; any storage mutation that could touch that generation's evidence acquires the same barrier across revalidation and mutation.
- Which barriers an operation must hold is **derived from the artifacts the plan actually touches**, not from a fixed list: the generations and run roots named by the planned actions determine the seams acquired. All acquisitions follow one order - run-activity leases, then P5 publication barriers, then P7 publication barriers - so P5 execution, P7 publication, and storage share a single cycle-free order.
- Generation supersession is not a liveness proof. A post-selection run that began under an older selected binding may still be executing, so its owner holds a run-activity lease for its whole write lifetime and storage waits for the owner rather than for a pathname to look old.
- The storage-operation lease serializes storage operations against each other. It does **not** serialize storage against P1-P7 writers, and it is never treated as if it did.
- No broad CampaignStore transaction is held across hashing, compression, or recursive scans.
- Where an owner exposes no race-safe reclamation seam, the affected artifact is retained.

- P3′s accepted publication-window evidence-graph fence remains valid substrate and is reused, not replaced.
- P7 durable `objects/` are ordinary protected evidence and are never an orphan-collection target.

5. Actions and tiers

`storage report`

Owner views supply semantics; bounded physical metadata supplies size. The normal report is **bounded**: it performs one `lstat` per owner artifact, never walks a subtree, and never reads or hashes an owner's O(member-count) topology manifest - it validates only the compact completion anchor. Its cost is therefore a function of how many artifacts the owners declare and not of how much bulk the campaign holds. It reports destructive eligibility as *potential*; the exact closed-subtree certification that authorizes a mutation happens at planning time and is the only thing that reads a full topology manifest. That applies to P7 released-attempt scratch as well as to P5 runs: a released attempt appears in the normal report as a retained container that still needs exact certification, so reporting never scales with attempt bulk and never implies that deletion authority has already been established. A directory's aggregate size is therefore reported as unknown rather than guessed, and the field says so (`size_scope = "unknown_without_deep_audit"`).

The report is a **complete census**: every known campaign family is accounted for, and any workspace tree no owner adapter claims is reported as ambiguous and retained rather than being pooled into a generic bucket or silently omitted. It reports owner and artifact class, current/historical/restart/cache/archive state, coverage semantics, potential reclaim per action, unresolved owners, owner-graph integrity failures, and protected inputs. The SHA-256 receipt cache is reported separately from CampaignStore authority.

Observational commands are side-effect-free

`report`, `report --deep`, `--dry-run` planning, `archive list`, and `archive verify` are observational. They do not create the workspace, the campaign state database, generation roots, or the storage control plane; they do not write acceleration receipts; and they do not deposit a report file in the campaign. The payload is returned to the caller and printed. Describing a campaign is never what brings it into existence, and an uninitialized campaign is reported as uninitialized rather than created.

Consequential storage planning is unavailable while any qualification attempt state cannot be authenticated. An active attempt's references can pin exact P5 checkpoints far outside the P7 tree, so an unreadable state is an unknown cross-owner retention edge; reporting names the exact state and reason, and planning refuses rather than guessing which artifacts it would have protected. Repairing the state restores planning.

Authentication is strict and root-bound, and it is performed by **one** authority that every storage-facing consumer reads - the census, the active-reference collection, the released/aborted/terminal classification, the retention fence, and reporting. A parallel permissive parser would let the local classifier call an attempt released while the owner graph was simultaneously calling it unknown. An attempt is authenticated only when the enumeration reached it without traversing a substituted namespace component, its state is a plain regular file read no-follow, its persisted `content_digest` recomputes exactly, its recorded identity equals the directory it was read from, **and** that identity equals the canonical identity derived from the qualification binding the state names. The directory name is not independent semantic authority: P7 derives attempt identity from the binding, so a state whose binding says it belongs elsewhere is not this attempt's state whatever it is filed under.

Namespace traversal is no-follow at every authority-bearing component, not only at the state file. `O_NOFOLLOW` protects the final name; a symlinked generation root or `attempts` container would otherwise let an entire foreign tree supply P7 state. Enumeration is by actual attempt *directory* rather than by state file, because an attempt directory with no state at all is exactly the case whose external references are unknown.

The descent is also **continuous**: from the accepted campaign internal root, each hop - the `qualification` family, the canonical `g<generation>` root, the literal `attempts` container, the attempt directory, and finally the state file - is opened relative to the descriptor of the parent that was already authenticated. A pre-check followed by a fresh pathname lookup would be two independent resolutions with a window in between; this is one, so an ancestor replaced mid-descent cannot be traversed. The generation namespace has exactly one canonical spelling, and a reserved name that does not use it (`g01`, `g+1`) is an integrity problem rather than another place to look for state. A descriptor-safe descent does not make a nested mount campaign-owned: the ownership boundary is unchanged.

Absence and ambiguity are different answers. A genuinely missing family or `attempts` container is ordinary "nothing here". A component that is present but cannot be authenticated - substituted, wrong-kind, unreadable, stale - and an entry enumerated but lost or replaced before it could be opened authoritatively are both unresolved authority, and they fail closed. Malformed persisted state is the same kind of answer rather than an escaping exception: a syntactically valid record missing a required field, or carrying an invalid container type, becomes one explicit unresolved result naming the attempt and the reason. Nothing is repaired by synthesis, and observational reporting stays available throughout.

Released-attempt authority is root-bound and generation-scoped. The v3 released-attempt proof records the attempt's canonical locator relative to the qualification family - `g<campaign_generation>/attempts/<attempt_identity>` - published from the owner's authoritative `PostSelectionBinding.campaign_generation`, and the strict storage reader independently recomputes it from the authenticated namespace it actually descended. A whole released attempt copied under another generation therefore carries no authority there, and the incomplete basename-only development form is diagnostic only.

That authority stays root-bound all the way to the mutation, and it is the *same* observation throughout. One strict descent produces the generation facts, the attempt state, the released-scratch topology, and - for a consequential action - the exact released-attempt proof and its certified node set. The storage-facing owner view is built from that result. It does not re-list `qualification/gN/attempts/<attempt>` through followable path APIs afterwards, because a second resolution would happily enumerate whatever a substituted ancestor points at by then; the expected generation-scoped locator is recomputed from the generation the descent authenticated, never from the parent names of a path.

Recursive deletion is descriptor-relative and no-follow **everywhere it is consequential** - the P7 released-attempt recursion, the generic cleanup recursion, and the certified-subtree recursion all descend through the same single no-follow directory acquisition. `shutil.rmtree` earns its symlink-attack resistance by descending through directory descriptors itself; that promise covers `rmtree`'s own implementation and is no protection at all for a separate walker. A recursion that classified a child as a directory and then reopened it by pathname could be redirected by an entry swapped in between, out of the authorized tree entirely, so each child is instead opened no-follow from the parent's descriptor and every unlink and rmdir is relative to it. The capability that guards these recursions is the directory-descriptor primitive set, not `rmtree`'s flag; where the platform cannot supply it the subtree is retained. The one remaining `shutil.rmtree` caller is a non-consequential utility that no cleanup path uses.

The destructive step holds one **live capability**, not a memory of one. Under the owner locks the apply path opens a released-attempt session: it re-acquires the strict namespace, requires the attempt root's `(device, inode)` to be the one the plan was made against, and then - on that still-open descriptor - re-authenticates the current attempt state, re-reads and re-binds the current released proof, and re-certifies the current typed topology. Mutation happens through that same descriptor: a proof-certified top-level regular file is unlinked with `dir_fd`, and a certified directory is removed by a bounded no-follow descriptor-relative recursion built from `os.open(..., dir_fd=...)`, `os.scandir(fd)`, `os.unlink(..., dir_fd=...)`, and `os.rmdir(..., dir_fd=...)`. `shutil.rmtree(..., dir_fd=...)` does not exist on the supported Python floor (`>=3.10`), and the floor is not

raised to avoid writing the recursion. Both the file and the directory case route through this one owner boundary; neither falls through to a generic absolute-path removal. Where the platform cannot supply those primitives the scratch is retained rather than removed by pathname.

A certification made on a descriptor that was then closed is not the final authority, and a planning snapshot's certified node set is not passed into the remover as if it were: between the close and the next open, the name can mean something else. One session serves every released action of the same attempt, so the authority is verified once per attempt and never lapses between verification and use, and the proof's typed-node lookup is materialized once per session and handed out read-only - rebuilding it per member would re-walk the whole proof for every target, and a writable view would let a caller widen the certified set after authentication.

Immediately before each member mutation, and through that same retained descriptor, the owner re-observes the **planned target identity**. Ordinary plan revalidation ran earlier and by pathname; an object swapped in afterwards under the same name and kind would otherwise inherit the plan's permission to delete it. The dimensions are the ones the plan already binds - `kind`, `device`, `inode`, `size_bytes`, `mtime_ns` - and the final boundary is never the weaker of the two checks. A missing target is `already_absent` and reclaims nothing; a present target whose identity differs is refused. The identity is **required**, not a caller convention: a released-member mutation with no plan-bound identity, or one missing any of those dimensions, is refused before anything is observed or removed. Without it there is no specific object the action was authorized against, and making the comparison optional would leave the boundary to be remembered by every future caller.

The capability is one-way. Once closed or invalidated it can never be spent again, and that guard runs *before* any syscall using the stored descriptor: the integer is not an identity, and the kernel is free to hand the same number to the next thing that opens a file.

A contradiction found at any member's mutation boundary is evidence about the whole attempt, not just that member - the premise every action of that attempt shares has just been seen to fail. So the capability is withdrawn, and the attempt's remaining planned members inherit an explicit no-change refusal without reaching the filesystem. Successful removal and already-absence are expected monotonic states and do not withdraw anything. Independent attempts are unaffected; their authority was never in question. A failed acquisition is likewise the attempt's answer for that execution rather than something retried per member, and there is no retry-until-convenient loop.

The plan is bound to the **exact released authority**, not only to paths and kinds. A released-attempt scratch action carries a derived identity over the authenticated attempt-state digest and the authenticated v3 proof digest, bound to the generation and attempt, and it rides the ordinary owner-state binding into the plan. A state and proof resealed to a different but equally valid release expose the same member names, kinds, and root inode; only this identity notices, and the old plan stales rather than authorizing the new release. It is derived on demand and never persisted.

Fresh certification treats the released proof as an **upper bound** on what P7 authored, not as a census that must still be complete. Every observed live descendant must be proof-recorded with the exact kind, and symlinks, special nodes, nested mounts, kind changes, and unrecorded nodes all refuse. Proof-recorded nodes that are gone are simply gone: an earlier action in the same cleanup, or an interrupted prior one, legitimately shrinks the live tree, and requiring equality would make correct multi-action cleanup invalidate itself after its own first success. Interrupted cleanup therefore stays resumable from the same unchanged proof.

Removal owners report a **terminal outcome**, not a boolean. `removed` completed the action; `already_absent` was terminally satisfied before this execution started; `refused_no_change` withheld the mutation with nothing changed; `partial_change_refused` unlinked some authorized members and was then stopped by a contradiction or a failure. Only the first two are completed actions, only mutations credit reclaimed bytes, and a partial makes

the execution `partial` even when it is the only action. Reclaimed bytes for a partial are what was measured before deletion - never the planned target size - and the semantic outcome is never inferred by parsing a reason string.

Removal and the fsync that makes it durable are two steps, so a failure can arrive *after* the disk already changed. So can a recursive removal that takes a subset of a tree and then cannot continue: a subset is gone while the container survives, and asking only whether the top-level pathname disappeared cannot see that state. Every removal owner therefore keeps a running account of what it has actually taken - each entry measured before its unlink and credited after it succeeds - and any later failure carries that account to the current action boundary, where the action is recorded before the failure continues upward. One recorder, owned by the executor, serves both the CLI cleanup engine and the default engine, so the two cannot drift into different truths. `removed_bytes` names directory entries actually removed under the runtime accounting metric; it is not a promise that crash durability was confirmed, and the durability failure stays explicit in the detail and propagates. A failure *before* the first destructive transition records no mutation and no bytes.

What establishes that account is the **transition itself**, never a later look at the pathname. The unlink primitive reports one fact - whether *this* call's syscall succeeded - and reports it immediately, before the durability step that can still fail. An owner that instead asked afterwards whether the name was gone would credit itself for a removal another actor performed, and would miss its own removal whenever another actor recreated the name; a target that was already absent under `missing_ok` was never removed by this call and is not credited to it. There is no compatibility fallback that calls an older signature and then invents the transition report it lost.

A file whose size and inode cannot be observed is **retained**, not removed. Deleting it and crediting zero would put bytes beyond recovery that the action can never account for, and with nothing else yet removed the outcome would read as "nothing changed".

Every recorded action carries its own reclaimed-byte figure, and the execution aggregate is exactly the sum of them. An aggregate alone cannot say which of several actions mutated, or explain two partial actions in one execution; the planned size is never substituted for what an action actually removed.

Mutation truth is recorded independently of that figure, because the two are different facts. Unlinking a zero-byte file, removing an emptied directory, and dropping one more hard link to an already-counted inode all change the namespace while crediting nothing. `mutated=true` with `reclaimed_bytes=0` is therefore a legitimate and meaningful record, not a contradiction. An owner that decided "did this mutate?" from the byte total would report those removals as no change - and would also skip the durability step it owes for entries that really went, since that step is gated on mutation rather than on bytes.

The execution's own terminal status follows the same rule: it is derived from what the action recorder captured, never from how control left the engine. An execution interrupted **before** any action changed anything is refused, not partial - calling it "partial after a strict subset of actions" would assert a mutation that never happened, and that is the sentence an operator reads after an incident. An interruption after a recorded mutation stays partial. Either way the audit is published before the failure propagates and nothing is rolled back.

Substantiated bytes use the planner's own convention: regular files only, counted once per `(device, inode)` across the whole action, so a file with several hard links is not counted several times. A fully removed nested subtree carries its measured amount up to a parent that later stops, because bytes the filesystem really gave back must not vanish from the figure an operator reads.

The guarantee this provides is **descriptor-pinned owner ancestry and fd-relative no-follow mutation under the supported-owner synchronization** - not an atomic inode compare-and-delete, which POSIX does not offer: directory-entry deletion is name-relative to a parent descriptor. What that buys is precise. An action already inside its mutation completes against the object it holds open, which is the object that was certified. Any action that re-acquires by name after a replacement finds something that is not that object and refuses. Authority

is therefore never transferred to a replacement tree - by symlink or by a same-shaped plain directory with a different inode - and a symlink, special node, wrong-kind replacement, unrecorded descendant, or nested mount remains a refusal boundary rather than a wider one.

A consequential destructive action acquires its target the same way, and from a **justified root of trust**. Cleanup does not re-open `path.parent` - or `path.parent.parent` - by pathname and treat the result as authority: a fresh multi-component resolution wearing a no-follow final component proves something about the object it opened and nothing about the object the plan authorized. The chain instead starts at the campaign workspace root the plan is bound to, which `StorageExecutor.run` holds the storage-operation lease and every touched owner's activity/publication barrier across, and descends componentwise from there, each hop opened relative to the descriptor of the parent that was already authenticated and put through the same opened-descriptor mount decision. It is the same discipline the P7 attempt acquisition already uses.

At the end of that chain the action spends the **plan's own target binding**. `PlannedAction.filesystem_identity` is compared - `kind`, `device`, `inode`, `size_bytes`, `mtime_ns`, exactly the dimensions ordinary plan revalidation binds - against the live object observed no-follow through the authenticated parent, immediately before the destructive syscall, and for a directory again against the *opened* descriptor the recursion will enumerate. This applies to the ordinary default single-file removal as much as to the recursions: revalidation ran earlier and by pathname, so without this a same-name replacement installed afterwards would inherit the old action's permission. An absent target is `already_absent` and credits zero; any bound dimension that differs is refused before anything is removed. `action.size_bytes` is separate aggregate accounting and never substitutes for the identity field. The owner's `root_identity` / `authority_identity` remain **independent** constraints checked on the opened authority-root and container descriptors; each may only narrow authority and neither stands in for the other. The public thin remover keeps an unbound convenience mode, and no consequential executor or production cleanup path can reach it.

The last destructive syscall gets the same treatment as the first. `rmdir` takes a *name*, not a descriptor, so a directory that has just been emptied can still be unlinked and recreated - or replaced by a symlink - before its own removal. The parent descriptor the descent authenticated therefore stays open through that boundary, and immediately before the syscall the entry is stat'ed no-follow relative to it and compared with the still-open child descriptor; only then does `os.rmdir(name, dir_fd=parent_fd)` run. A consequential recursive root is never removed by absolute pathname once descriptor authority exists. **Persisting that directory entry belongs to the same capability**: the retained parent descriptor is fsynced after the fd-relative `unlink`/`rmdir`, not closed and re-opened by pathname, because a replacement installed at the parent's name in between would otherwise be the directory that got persisted. A parent-durability failure after a successful transition is a structured partial mutation carrying this action's exact bytes; it is never a no-change refusal. Only the irreducible window between that comparison and the syscall is outside the guarantee.

An individually-authorized member of a retained container is reached the same way: the container is acquired relative to an authenticated parent descriptor, every intermediate directory on the way to a nested member is opened no-follow and put through the same opened-descriptor mount decision, and the member's observation and mutation are descriptor-relative. Deletion authority additionally requires the owner's **typed** evidence for that member. A bare path with no certified kind authorizes nothing and is retained; defaulting a missing type to "regular file" would let the absence of authority stand in for permission, which is the one substitution no later check can catch.

Descriptors acquired by these descents are released exactly once on every path - success, refusal, partial, and exception alike - so a bounded loop of contradictions cannot accumulate them. Ownership is explicit in the shared no-follow acquisition itself: it either hands the descriptor to its caller and never touches it again, or releases it exactly once behind the namespace/authentication failure it decided, so a failing cleanup close can never provoke

a second close of a number the kernel may already have reissued. A failed released-attempt session acquisition follows the same rule - the owner, root-identity, release-authority, state/proof or topology refusal is materialized first and the descriptor is ranked and released afterwards, rather than through a raw `finally` close that could cancel the refusal the caller records. A close that fails is neither swallowed nor allowed to displace the truth: while a primary product failure is propagating the close failure is logged as secondary evidence, because the primary is what carries the mutation account; when the close is the only failure it becomes the outcome, a partial if the action had already removed something and a plain no-mutation failure if it had not. A released-attempt capability is withdrawn and its descriptor number cleared *before* the kernel close, so a failing close can never leave a session that still looks spendable, and the primary-versus-secondary ranking belongs to the caller that knows whether a product failure is already in flight rather than to the session inspecting ambient exception state.

The consequence of that ambiguity is deliberately blunt and **workspace-wide**. Because the lost references routinely name artifacts outside the P7 tree, the qualification retention fence enters an explicit ambiguity state that denies destructive authorization for every campaign-managed path until the state is repaired. The fence only ever denies - it grants nothing - and it sits behind the owner-graph gate as defense in depth, so a destructive path that somehow skipped planning still refuses. Read-only reporting stays available throughout: it is exactly when the boundary is refusing everything that an operator needs the report naming what is wrong.

Observation is an **invocation-scoped capability**, not a property of the first store the command happens to open. It propagates to every nested owner helper and into every worker thread the invocation spawns, so a helper cannot escape it by calling an ordinary default-creating constructor from a worker. It is enforced at the owner boundary as well as declared: an observational campaign-state open uses a genuinely read-only SQLite connection, and a write attempted through it is refused before anything is committed.

Nothing process-global is toggled to achieve this. Receipt lookups are read-only in an observational context and receipt writes are simply skipped, so an observational report running beside a legitimate consequential operation neither writes anything itself nor disables or redirects that operation's own caching.

`storage report --deep`

Explicit exact recursive physical accounting, symlink and ownership inspection, and largest-artifact detail. Read-only.

`storage cleanup --tier safe`

Zero scientific, restart, qualification, locked, and acceleration-cache capability loss. Candidates are only artifacts an owner has positively released: external record payloads no campaign state row references and that are outside the publication window; attempt-local bulk of a P7 attempt the owner has released, excluding the attempt record itself, whose terminality is monotonic; and abandoned storage-native staging with no open journal. Safe performs no acceleration-cache eviction.

`storage cleanup --tier cache`

Safe plus eviction of cache/index state whose owner can prove exact reconstruction *at that moment* **and** can prove that no live consumer is depending on it right now.

Reconstructibility alone is not sufficient. The normalized frame cache is reported as exactly reconstructible - the DATA2 source catalog resolves, the cache manifest binds that exact catalog digest, and every per-run source identity and control signature matches, so a rebuild costs one source read per run and reproduces the identical authenticated cache - but P1 exposes no consumer/builder liveness seam, so evicting it could race a reader mid-campaign. It is therefore **retained by both tiers** and reported as `cache_reconstructible = true, cache_evictable = false`. Positive eviction is unlocked by adding a real P1 liveness seam, not by relaxing this rule.

The SHA-256 receipt store is likewise accounted as reusable cache and retained by both tiers: it is the acceleration cache the running storage operation is itself writing to. Anything an owner cannot certify is retained, and a `cache` action over a campaign with no certified family is legitimately a no-op.

`storage cleanup` and campaign-state maintenance

Bounding the campaign store's diagnostic events and rewriting its SQLite file are **two separately planned actions**, not one decision and not a free tail call on the end of a cleanup. Maintenance appears in the plan as its own action or it does not happen; a refused or empty cleanup can never piggyback either mutation.

Excess diagnostic events authorize **pruning only**, and the resolved retention bound is executed exactly - there is no hidden execution floor that would make the plan, the policy identity, and the audit record describe a retention that never happened. Pruning is a small owner-local transaction that takes the write lock up front, so it serializes against any other campaign writer rather than assuming one process owns the database. A **rewrite** exists as its own action only when a fresh observation already satisfies the configured reclaimable-byte or reclaimable-fraction predicate. At execution it re-establishes that predicate and its temporary-space admission *inside a cross-process exclusion* that every campaign-state writer participates in - **including writable construction**, whose schema bootstrap is a real write - and it holds that exclusion through the rewrite itself. The exclusion is one gate per database shared by every store instance in the process, its reentrancy belongs to the acquiring thread rather than to an object, and the advisory lock beneath it is what a second process blocks on. That lock file is campaign-store owner infrastructure: it is never a cleanup, archive, or dedup target, because unlinking a held advisory-lock pathname would split the serialization domain between the old inode and a freshly created one. Measuring free space and then queuing for the database would be a race, not a decision: a second process - normally a second CLI invocation, which no in-process mutex can see - can commit in between and consume exactly the space the rewrite was authorized by. The exclusion is released on success, refusal, and failure alike. Free pages that the prune itself created do not widen the prune into a rewrite: that decision belongs to the next fresh maintenance plan, measured on its own evidence.

Results and audit records distinguish `events_pruned` from `vacuum_performed`. A skipped or failed maintenance is reported truthfully and changes nothing about whether the file actions succeeded.

`storage archive`

A reversible authenticated cold representation of owner-declared historical reproducibility bulk. See section 7.

`storage deduplicate`

Owner-certified immutable deduplication. See section 8.

Retired tiers

`recompute` and `compact` are retired consequential-loss tiers. They are rejected by name; intentionally lossy history pruning requires a separate explicit product decision.

5b. Recursive authority: closed subtrees and containers

A directory-level owner view carries one of two explicit coverage semantics, declared by the owner:

- **closed subtree** - the real owner certifies, from its own authenticated record or exclusive-writer contract, that every traversable descendant belongs to that artifact. The certification bounds what may be acted on: a descendant the owner did not record is a contradiction that withholds authority over the whole artifact rather than being absorbed into it. A recorded member that is *absent* is not a contradiction, because content may legitimately have left the tree into a cold archive.
- **container / open subtree** - the directory is owner-known but its descendants require individual owner views. Unknown descendants stay ambiguous and retained, and the container itself is reported but never acted on.

Certification comes from the owner, never from a filename extension, an age, a stage name, or a storage-authored pathname convention. Concretely:

- a post-selection run root is closed only under P5′s own terminal completion proof, because the run directory is delegated to the configured trainer and P5 alone can say what it produced there. That proof is deliberately two records: a **compact completion anchor** that is O(1) to validate and is the publication commit point, and an immutable **topology manifest** naming every node the run produced. Both are versioned and self-authenticating; a tampered, copied, or self-inconsistent proof makes the run non-certifiable rather than widening what it appears to own, and the superseded single-file development format grants no authority at all;
- a released P7 attempt publishes a versioned typed topology proof bound to the exact released state it was published for. The proof is written first and the released state second, so the state is the commit point: a crash in between leaves a proof bound to a state that is not current, which grants nothing. An aborted attempt that legally reopens as active therefore invalidates its own release proof for free. Storage and P7 take the same per-attempt state lock, so a reopen and a storage operation on the same attempt can never interleave in either order. A repeated terminal release validates the retained proof against the exact terminal state under that lock and reuses it; it never rebuilds one by rescanning a tree storage may already have depleted or something may have tampered with, and a missing, superseded, self-invalid, or cross-field-contradictory proof fails closed with the scratch retained. Every top-level node has to be one the proof recorded, of the recorded kind, before it can be exposed as reclaimable at all, and the superseded development record grants no authority;
- the campaign store's externalized record area is closed by exclusive-writer contract, tightened to exact manifest agreement for a sharded record;
- a superseded target-size execution root records no member manifest, so it is honestly a container: storage reports it and never acts on it.

The recorded topology is **typed and covers directories as well as regular files**. A recursive delete makes directory nodes disappear too, so an unexpected *empty* directory that no recorded node mentions would otherwise be swept away by an action nobody authorized to remove it. Kind is part of the proof, not decoration: a recorded regular file replaced by a directory at the same relative name - or the reverse - is an ownership contradiction, and a comparison that had already reduced both sides to path strings could not see it. A symlink or special object is never made owned by appearing at a familiar name.

Every observation on that path is **no-follow**. Nodes are classified with `lstat`, and the owner's own authority records are opened with `O_NOFOLLOW` and confirmed to be regular files by `fstat` on the opened descriptor - not by a separate `lstat` that a rename could invalidate before the read. A symlink substituted at a completion anchor, topology manifest, attempt state, or released-attempt proof therefore grants nothing, and its target's bytes are never consumed as owner proof.

Recursive cleanup, archive collection and hot reclamation, dedup enumeration, and restore planning recurse destructively only through a freshly revalidated closed-subtree contract, descending descriptor-relative with no-follow semantics and evaluating mount trust directly on opened directory descriptors. An unexpected descendant that appears between planning and apply reduces or refuses the planned action rather than being deleted with it, and an archive manifest records only owner-certified members.

One narrow exception exists and is named explicitly: a directory the storage subsystem is the *sole writer* of - its own `.mdstats/storage/staging` scratch - is closed by that exclusivity rather than by an enumerated member set, because enumerating a set from the very tree in question would be circular. It never applies to a directory another component writes into.

5c. Cleanup semantic classes and the default execution domain

Cleanup is not one destructive operation. Removing a released P7 attempt member spends a live attempt session's proof, release, and root binding; emptying an owner-scoped container spends typed member certification, retained

members, and the owner's root/path identity; unlinking an orphan record file spends nothing beyond the plan's own target identity. Those are different authorities, so *which* implementation may act is a semantic decision, and it is made in exactly one place.

Cleanup semantic authority is also **invocation-local to the cleanup action family**. `StorageExecutor.run` is a shared authorization shell that archive, deduplication, restore, and cleanup all pass through, and revalidation proves only that the executor's resolved policy and the plan's policy agree *with each other* - never that the actions they agree on belong to the family the invocation authorized. So the canonical cleanup owner refuses any plan whose resolved action is not `cleanup` **before it inspects or dispatches a single action**, and both the production cleanup engine and the executor's default engine reach per-action semantics only through that one gate. The gate is total: an empty plan, a maintenance-only plan, and a plan carrying an owner-released removal are refused identically under an archive, deduplication, restore, or report policy, because an empty wrong-family execution that settled `complete` would report that an engine had performed an operation it does not implement. A genuinely empty `cleanup` plan is untouched and remains a valid, successful, non-mutating no-op. The exported single-action classifier is bound the same way: under a non-cleanup policy it returns `invalid` rather than any positive cleanup class, so the family rule cannot be bypassed by reaching the classifier directly.

Within a cleanup invocation, every cleanup action resolves, from the **fresh post-revalidation snapshot while the storage-operation lease and every touched owner's activity/publication barrier are still held**, to exactly one class:

- **exact authorizer** - the owner names its own authorizer, and only that owner's implementation may mutate the artifact. An authorizer no implementation exists for is unsupported, never generic;
- **owner-scoped subtree** - a directory artifact whose removal spends typed member, retained-member, and owner root/path authority;
- **generic leaf** - a non-directory leaf whose current owner view exactly matches the action's artifact and path, whose action kind that owner still grants, and whose mutation needs nothing beyond the plan-bound target identity and the synchronization already held;
- **maintenance** - campaign-state maintenance, realized by its own owner engine;
- **invalid** - anything that cannot be positively established as one of the above.

The domain is **positive**. A new owner field, coverage mode, or authorizer does not become a recursive delete by failing to match an earlier branch; it becomes invalid, and invalid never mutates. That negative-fallthrough shape is precisely how owner authority disappears silently, so no cleanup path is permitted to define these classes a second time: the production cleanup engine and the executor's optional default engine consume the same classification and differ only in which classes they implement.

An action is also bound back to owner semantics rather than to a path alone. Its `artifact_id` must resolve to an owner view at exactly the path it targets, and the current view must still grant that action kind - safe reclamation for a removal, and certified reconstructibility plus owner evictability under the cache tier for an eviction. A valid artifact identity, a passing physical ownership boundary, and a valid plan-bound filesystem identity are independent constraints and none of them substitutes for this binding.

Each cleanup engine's declared supported domain and its actual dispatch are **one closed set**. Generic destruction is reached only from an explicit positive `generic leaf` branch, never as the residual of the owner-specific tests above it; a class that reaches a dispatcher without a handler raises a typed engine/domain failure before any destructive work. The two definitions cannot drift, because the set an engine preflights as supported is the same declared set its dispatch branches on, and that equality is checked directly. Widening an accepted domain without adding a handler therefore fails closed rather than reappearing as a generic delete.

`StorageExecutor.run` without an engine has the **generic-leaf class as its entire destructive domain**. Engine/domain incompatibility is decided over the **whole plan before the first transition** and is independent of action

order, so no convenient prefix of a plan is spent before an unsupported later action is noticed. Such a plan completes no action: it is non-mutating, credits zero bytes, and is published in the durable audit as a refused execution naming the engine/domain incompatibility - which is distinct both from a stale plan and from an owner refusing a particular target at its mutation boundary. Ordinary per-action refusal at the mutation boundary keeps its existing settlement semantics.

6. Resolved storage policy

One canonical resolved policy is shared by every CLI, config, and API entry point. It normalizes aliases before hashing, so equivalent spellings produce one identity, and it binds the requested action and tier, storage and scratch safety reserve, cache eviction limits, SQLite compaction thresholds, archive codec/level/expansion bounds, dedup realization and minimum file size, I/O worker limits, deep-audit bounds, lease timeout, and audit retention.

Policy identity is **action-scoped**: it hashes only the fields the invoked action actually consumes, so changing an archive codec cannot invalidate an unapplied cleanup plan and cannot change a cleanup's identity. Every public policy field is consumed by at least one action; a knob no action reads would be a false contract and is rejected at import.

- Changing a material policy value invalidates an unapplied plan for the actions that consume it.
- A storage policy change never invalidates a P1-P7 scientific identity.
- Presentation-only options are deliberately outside the policy identity.
- Free bytes, inode headroom, and observed saturation are **execution observations** recorded on the plan, not policy. A changed disk causes admission revalidation, not scientific invalidation.
- No environment variable may widen deletion or archive authority; the resolver reads none.
- An unrecognized `[storage]` key is rejected rather than ignored.

7. Cold archive v2

Archive replaces hot bytes only when **all** of the following hold:

1. the owning artifact is historical or otherwise explicitly owner-declared cold-replaceable;
2. no current or restartable dependency closure requires its canonical hot representation;
3. no current owner resolver or currentness validator directly requires the canonical hot file;
4. explicit restore is sufficient to regain the promised historical capability;
5. the archive is authenticated and cataloged before any hot deletion.

Archive is **not** a transparent virtual filesystem. No P1-P7 loader is given an implicit "if missing, read the storage archive" fallback by this subsystem.

Selection, representation identity, and the restore plan

`--root` may **narrow** a selection into an eligible owner artifact; it may never **widen** it to an ancestor. A requested root that is an ancestor of an eligible artifact is rejected outright rather than silently reinterpreted, because an ancestor sweeps in siblings no owner released.

An archive's identity binds its **representation**, not only its logical content: the codec, the compression level, and the serialization contract are part of the identity. Re-encoding the same logical members under a different codec produces a distinct retained representation, and a failed re-encode can never invalidate the representation already retained.

A restore is an **exact owner-bound plan** over named members and containers, computed before anything is staged. `--dry-run` computes the same plan and installs nothing.

Restore never mutates a pre-existing container

Restore distinguishes a directory it creates from one that was already there. A directory this restore creates may receive the archived owner-certified mode and metadata. A **pre-existing** directory is never `chmod`-ed, replaced, or otherwise metadata-mutated merely because the archive contains a directory entry with the same path; only its own owner changes it. Its timestamp may move for the ordinary reason that its entries came back.

A pathname is not a directory. The restore plan binds the exact `(device, inode, type)` of every existing parent it will install through, and verifies each one again immediately before creating or installing anything. Replacing a planned parent with a *different* ordinary directory at the same path - same mode, same type, not a symlink - refuses the restore rather than silently redirecting the installation. Parents this restore creates itself are validated by its own creation and postcondition chain instead.

Protected reauthentication

A reclaim or restore plan binds the exact retained representation it intends to consume: the representation identity, the create-once catalog fields, the manifest content digest, and the blob locator, digest, and size. Authenticating that representation while *planning* is not sufficient, because the bytes a reclamation is about to trust could be replaced or truncated afterwards.

So the exact catalog entry, manifest, and blob are re-read and re-authenticated **inside the protected consequential window** - after the storage-operation lease and every owner seam are held, and before any hot member is removed or any member installed. A mismatch removes nothing and installs nothing; the operation is refused and re-planned against current retained authority.

That check is race-closed because every supported product path that creates, replaces, removes, retires, or operationally updates retained archive control state acquires the same storage-operation lease first; read-only list, verify, and report do not. This is not an OS security boundary: a process that deliberately ignores package ownership and rewrites campaign files is treated as corruption and detected at the next protected authentication point.

Locator containment

The archive catalog and its blobs live under one storage-owned authorized root. A manifest carries a canonical identity-owned relative locator, never an arbitrary filesystem path. An absolute locator, `..`, an empty or alias-normalizing component, a symlink escape, or a locator resolving outside the authorized root is rejected. A manifest field never authorizes reading an external file, even when those bytes satisfy the recorded digest. Member path safety is enforced separately and is not replaced by validating the outer locator. Restore from a user-supplied external archive is not a supported feature of this package; adding it would require an explicit trust/import contract.

Bounded verification and extraction

Archive bytes are authenticated-but-untrusted until every check passes. Before and during extraction the subsystem enforces: supported schema and codec only; canonical workspace-relative member paths with no absolute path, `..`, empty component, normalization alias, or post-normalization duplicate; regular files and directories only, rejecting symlink, hard-link, device, FIFO, and socket members; manifest member-count and total-expanded-byte admission; a hard per-member size bound applied *while streaming*; a cumulative extracted-byte bound; a compressed-to-expanded ratio bound sufficient to refuse decompression amplification before extraction; a campaign-owned staging root with no traversal through archive-created symlinks; exact digest and size authentication before install; and no implicit overwrite of conflicting authoritative bytes.

Durable publication ordering

```
write/stage
 -> flush + fsync content
 -> atomic publish
```

```
-> persist the parent directory entry where supported
-> authenticate the published bytes
-> publish the dependent manifest/catalog/terminal receipt
```

Publication truth is established at the atomic replace, not when the helper returns. Everything after the replace - the parent-directory fsync, the read-back, the digest authentication, the JSON reparse - runs while the canonical name already resolves to the new bytes, and any of it can fail. That fact cannot be recovered afterwards either: the target may have pre-existed, or another actor may have republished it, so observing a file there proves nothing about this invocation. The publication primitive therefore signals the transition the instant its replace succeeds, and the execution records it there. Archive creation carries a monotonic publication phase - `blob_published`, then `manifest_published`, then `catalog_published` - each advanced at its own replace and never regressed by a later failure, and byte credit is a separate claim that waits until the amount is substantiated.

Hot deletion follows an authenticated archive and a durable catalog, then a fresh owner/dependency revalidation, then removal of only still-authorized hot members, then a truthful terminal status. Restore is bounded authenticated staging, durable publication into canonical hot paths, final owner/content authentication, and only then a terminal receipt. Because a nonterminal restore journal is retained recovery authority, publishing it is a real storage mutation in its own right: a restore that publishes its journal and then fails is `mutated`, recorded at `journal_staging_published`, even though no destination byte has changed and `restored_bytes` is legitimately zero. The terminal replacement advances the phase to `journal_terminal_published` at its own replace, and a terminal receipt is never claimed when that replacement did not occur. Destination container creation and member replacement continue to record their own transitions and their own restored bytes independently of the journal. The subsystem does not claim power-loss durability stronger than the filesystem provides; where a directory fsync is unavailable the content flush and atomic rename still hold.

The catalog is identity-keyed. There is no `latest` authority. Restoring bytes never promotes historical evidence to current.

8. Immutable deduplication

Deduplication is **direct**: byte-identical members share one inode among themselves. There is no persistent content-addressed store, because a second durable copy of campaign bytes would be a second retention authority with its own lifecycle, reference counting, and failure modes. Removing the last alias therefore releases the inode naturally, with nothing left behind to garbage-collect.

Eligibility is `owner-certified immutability + exact content identity + owner-certified metadata compatibility + closed link ownership + filesystem realization support + race-safe replacement`.

The pre-rename hardlink is staged inside the storage subsystem's own `.mdstats/storage/staging`, keyed by operation identity, and never inside the owner's run directory - a hard crash between the link and the rename therefore leaves storage-owned residue that the existing abandoned-staging lifecycle retires, rather than an unrecorded descendant that would permanently block the very reclamation it interrupted. Staging and the destination must share a filesystem; if they do not, the group is refused rather than downgraded to a copy or an unowned temporary. Whether staging is abandoned is established by the storage-operation lease and the restore journals, never by a process id, an age, or a pathname convention, and cleanup only unlinks staged names: writing or changing metadata through a staging hardlink would mutate the canonical inode itself.

Closed link ownership means the canonical member's link count is fully accounted for: either it has exactly one link, or every one of its links is a known member of the group being deduplicated. A file that is already hardlinked to something outside the campaign is never chosen as the shared canonical inode, because relinking to it would silently give an external file authority over campaign bytes.

- File type and required mode, ownership, and other owner-required metadata must match; equal bytes with divergent metadata are never hardlink aliases.
- A deduplicated family must have no accepted in-place content or material-metadata writer. Only superseded-generation roots participate, because every P1-P7 writer writes into the current generation.
- Replacement is a single atomic rename of a fully linked temporary, so an interruption can never leave a temporary path accepted as canonical.
- Cross-device or unsupported filesystems retain duplicate bytes without a correctness failure.
- Mutable SQLite state, active attempt scratch, and any owner-ambiguous file never participate.
- Dedup changes inode and ctime and therefore invalidates stat-keyed receipts. That is a cache miss and a revalidation, never a scientific state change.

9. Storage-native control plane

The subsystem owns durable state of its own under `.mdstats/storage/`: an identity-keyed archive catalog, archive manifests and blobs, restore journals, a bounded execution audit, operation-serialization state, and restore staging.

- The catalog, manifests, blobs, and journals required to locate, authenticate, resume, or restore a retained cold representation are never deleted or archived away by any action, including audit pruning.
- The execution audit is diagnostic evidence with bounded retention. Losing an old record cannot invalidate scientific currentness, and an incomplete operation is never recorded as complete.
- Restore journals are bounded: a nonterminal journal is recovery authority and is never pruned, while terminal journals are diagnostic evidence retained to a configured bound.
- Catalog fields that establish what a retained representation *is* are create-once. A rewrite that would change an immutable field is rejected; only operational fields may be refreshed.
- Operation-serialization state is operational liveness only. A crashed holder's advisory lock is released by the kernel; recovery never infers authority from a PID, hostname, or pathname.
- No control-plane record carries a scientific currentness decision, and none can make a historical owner artifact current. Plans, audits, and manifests carry no secrets or machine credentials.

10. Admission

Storage does not introduce a second, weaker free-space floor. The campaign's accepted `[execution].minimum_free_disk_gib` reserve and the `[storage]` safety reserve compose as the stricter of the two, because a reserve is a floor. Before a material storage operation the subsystem bounds free bytes against that reserve, inode headroom, and the operation's **peak** temporary amplification — a staged archive blob before reclamation, a restore staging tree plus its installed copy, a dedup temporary link, a SQLite `VACUUM` rewriting the state database beside itself. An admission failure refuses the operation before any mutation and leaves the campaign exactly as it was. Storage pressure never changes target membership, precision, epochs, seed or qualification population, timestep, acceptance thresholds, or a locked policy. I/O concurrency is controlled independently of CPU worker count.

11. Interruption and terminality

Every multi-action operation has an explicit terminality contract.

- **Cleanup**: each removal is independently owner-authorized and race-safe. A crash after a subset is acceptable because each completed removal was individually safe. No terminal `complete` audit is published until every action in that execution reaches a verified terminal disposition. A retry re-inventories and re-plans rather than reusing the old remaining set. Rollback of already-safe deletions is not required.
- **Dedup**: each replacement is exact and idempotent; a retry reauthenticates both the canonical member and the content object before linking.

- **Archive**: archive bytes without an authenticated catalog never authorize hot deletion. An authenticated catalog may truthfully coexist with still-hot members after an interruption; the catalog records that state, and a retry reconciles which members remain and removes only those still authorized under a fresh owner-bound plan.
- **Restore**: partial staged or installed state is never accepted as complete. The terminal receipt follows final canonical-byte authentication. A retry is idempotent for already-present identical bytes and fails closed on conflicts.

Every **normally successful** applied consequential path appends exactly one truthful durable audit record, and a read-only path appends none. An interrupted operation is recorded as `partial` if and only if persistent state changed (`mutated` is true), and `refused` if interrupted before any action mutated anything, never as `complete`. The stored record states its own successful publication, so durable evidence of a successful operation never contradicts itself.

Publication and bounded retention are one serialized lifecycle under the storage-operation lease. Retention reads the whole stream and replaces it, so an append that slipped in between would otherwise be rewritten away moments after being reported as published. Retention also refuses to rewrite over a truncated or unauthenticated stream - damage is surfaced, not overwritten - and a retention failure is reported separately: it never unpublishes a record that was written, never rolls back the mutation, and is simply retried by a later operation.

The audit is diagnostic evidence, not scientific authority, so its own storage can fail. When durable audit publication fails the mutation is **not** rolled back and no record is fabricated; instead the outcome is explicitly degraded rather than reported as ordinary success. The status itself carries the difference (`complete_unaudited`, `partial_unaudited`, `refused_unaudited`), the result reports the publication failure, and the CLI says so plainly. This is deliberately pessimistic rather than precise: after an arbitrary write or `fsync` failure a complete record may or may not have reached the file, and the subsystem does not promise a proof of absence it cannot have. What it does promise is that a caller is never told an operation was audited when publication reported failure. Retry and reconciliation start from actual current filesystem and owner state; a destructive action is never replayed merely to manufacture a missing audit record.

12. Production qualification disposition

Routine implementation requires bounded functional, restart, and integrity tests plus representative storage/I/O measurement. Real campaign external-DFT qualification, long target-machine GPU production qualification, and environment-specific HPC storage qualification remain deferred and are not claimed by this specification.